Complex Magnitude API

Kenneth Martin

March 25, 2020

Abstract

A very concise desription of the functions used.

Functions

cascadeClass.m

```
classdef (ConstructOnLoad = true) cascadeClass < handle
cascadeClass defines a class to hold Cascade Filter instantiations
The cascadeClass object contains biquads in cscFltrSctnClass objects
It also has a function for evaluating the frequency response of
complex cascade filters plus a number of other utility functions.
sctns: an array of cscFltrSctnClass objects which contain first or
    second order sections.
size: the number of sections
obj = cascadeClass(sctn): make a new cascadeClass object, sctn can be
empty, or a cascadeClass obj, for copying a filter, or
    cscFltrSctnClass which gets added as the first section
obj = addSctn(obj, sctn): add an additional section
gn = freqEval(obj, f): evaluate the cascade filter as frequencies f
    (in real Hz)
gn = scaleFltr(obj, wp): scale gains of sections so gain peaks are
    all 1 (f-inf scaling)
sys = getSystem(obj): update and return the overall system in a zpk
    system obj
sys = fshftFltr(obj, fshft): freq shift cascade filter; returns
    shifted zpk obj
sys = fsclFltr(obj, sclFctr): freq scale cascade filter using
    scaleFltrD.m
plotGn(obj, wp, ws, minY, maxY): plot the response of the cascade
    filter using plot_drsps.m
out = sim(obj, xin, delta_f): simulate the cascade filter for
    complex input signal xin, after optionally frequency shifting
    filter (useful for filterbanks)
```

cscFltrSctnClass.m

```
classdef (ConstructOnLoad = true) cscFltrSctnClass < handle
The cscFltrSctnClass is a class for a section or biquad of a digital
    cascade filter
The sections are first or second order. The sections ar stored as a
    zpk model and also
as zeros, poles, and k factor for easy Whenever any sub components
    are changed,
the zpk model is updated. There are a number of utility functions
    such as frequency
analysis, difference-equation simulation, etc.
obj = cscFltrSctnClass(sys): make a new cscFltrSctnClass initialized
    to sys (a zpk
system obj). If sys is a cscFltrSctnClass object, a copy is returned.
obj = setSys(obj, sys): change the system to a new zpk sys
obj = setZ(obj, z): change the zeros, and update sys
obj = setP(obj, p): change the poles and update sys
obj = setK(obj, k): change the k factor and update sys
mxGn = getGain(obj, wp): find the maximum gain aroung wp = [wp1, wp2]
obj = setGain(obj, gain, wp): set the maximum gain
gn = freqEval(obj, f): evaluate gain at frequencies f (freqz() is
    used for evaluation)
out = sim(obj, xin, delta_f):simulate the filter section for complex
    input signal xin,
after optionally frequency shifting filter section by delta_f(useful
    for filterbanks)
```

chckEqlOrdr.m

```
isEqual = chckEqlOrdr(ply, fun) checks if polyClass object ply is
    identical to fun. It's used to possibly terminate finding roots
    of a function early when the fun is itself a polynomial having
    fewer roots than expected.
```

chooseX.m

```
chooseX(func, inArry) applies min(func) to inArry
and returns the indx and the array with the min removed
It's a utility function used in grouping zeros and poles
when designing a digital filter
```

cleanRoots.m

```
rts2 = cleanRoots(rts, tol) sets any real or imaginary parts of
    vector rts having an abs() value less than tol to 0. tol
    defaults to 1e-5. Components with absolute values larger than
    tol are unchanged.
```

dAs10_dz.m

```
h = dAs10_dz(wsz,As,w) finds derivate of stopband specs using simple
interpolation of specifications in log10 to the transformed
variable z
```

design_dig_filt.m

```
[H, E, F, P, e_] = design_dig_filt(p,px,ni,wp,ws,as,ap,type) Design
    a discrete-time complex IIR transfer function.
p: Initial guess at finite loss poles; actual values not important
    as long as they are in the stop-band; this spec is mostly used
    to decide the order.
px: fixed poles; often used to have a loss-pole at dc or the carrier
    frequency.
ni: number of poles at infinity of the continuous-time prototype.
    They result in loss-poles at half the sample frequency
wp: the predistorted pass-band frequencies in radians. If the final
    desired passband frequencies (in Hz) are [fp1 fp2], then uses wp
    = 2*tan(pi.*[fp1 fp2]).
ws: the stop-band frequencies in radians; at least two, but there
    can be more than two; the number of frequencies should match the
    size of as; all loss-pole frequencies (p, and px) should be in
    the stop-bands
as: the specifications for desired attenuation at each stop-band
    frequency; the actual attenuation is determined by the order and
    how wide the transition region is, but generally the DIFFERENCES
    in attenaution at the ws frequencies is determined by the as
    specifications. as values should all be positive
ap: the desired pass-band ripple in dB; normally this spec. is met
    exactly.
type: 'monotonic' for a maximally flat pass-band and 'elliptic' for
    an equi-ripple pass-band
```

disp_margind.m

```
[margin, smin] = disp_margind(sys,ws,as,wp,e_,type) prints stop-ban
    margins, that is the differences between the stopband loss and
```

```
    th interpolated specs at the frequncies specified by the vector
    w. As should be in dB. The specification frequencies are in the
    transformed domain.
```

dlogKK_dp.m

```
h = dlogKK_dp(sys,px,s) is a program for calculating the derivative
    of the log of the magnitude of system Kz_ with respect to the
    los poles.
sys: the LTI object.
s: the frequencies of the minima
h: the returned matrix has one row for each minima frequency and one
    column for each moveable loss-pole
```

dAs10_dz.m

```
h = dAs10_dz(wsz,As,w) finds derivate of stopband specs using simple
interpolation of specifications in log10 to the transformed variable
    z
```

dsply.m

```
dsply(X0) is a utility function used during debug to simplify
    looking at a system object it is just a cover for sortZPK to be
    easier to type. z, and p are returned as vectors sorted on the
    jw axis
```

dsplyRes.m

```
dsplyRes(X0) is a utility function to display the residues of a
    system object. It is primarily used when debugging ladder
    removals
```